Personal Information

Name: **Dies de haan**

StudentID: **15879062**

Email: **dies.de.haan@student.uva.nl**

Submitted on: **23.03.2026**

Data Context

In this section you should introduce the datasources and datasets which you will be working with. Explain where they are from as well as their domain. Give an overview of what the context of the data is. You should not spend more than 1 to 2 paragraphs here as the core information will be in the next section.

Data Description

Present here the results of your exploratory data analysis. Note that there is no need to have a "story line" - it is more important that you show your understanding of the data and the methods that you will be using in your experiments (i.e. your methodology).

As an example, you could show data, label, or group balances, skewness, and basic characterizations of the data. Information about data frequency and distributions as well as results from reduction mechanisms such as PCA could be useful. Furthermore, indicate outliers and how/why you are taking them out of your samples, if you do so.

The idea is, that you conduct this analysis to a) understand the data better but b) also to verify the shapes of the distributions and whether they meet the assumptions of the methods that you will attempt to use. Finally, make good use of images, diagrams, and tables to showcase what information you have extracted from your data.

As you can see, you are in a jupyter notebook environment here. This means that you should focus little on writing text and more on actually exploring your data. If you need to, you can use the amsmath environment in-line: $e = mc^2$ or also in separate equations such as here:

$$e = mc^2 \text{ where } e, m, c \in \mathbb{R} \tag{1}$$

Furthermore, you can insert images such as your data aggregation diagrams like this:

image

Data Sources

This study uses multiple datasets:

Roboflow datasets (training support)

- Asbestos dataset (~1331 images)
- Crack detection dataset (~1551 images)
- House damage dataset (~3570 images)
- Paint damage dataset (~67 images)
- Mold dataset

DGW dataset (evaluation) not in EDA

- ~2000 indoor housing images
- Real-world variability (lighting, angle, clutter)

Key challenge

Datasets differ in:

- Label definitions
- Annotation style
- Instance density
- Presence of "no damage"

In [2]:
```
import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from collections import Counter
from roboflow import Roboflow
```

Data Loading

getting api key

In [3]:
```
def load_api_key(path="apikeyroboflow.txt"):
    with open(path, "r") as f:
        return f.read().strip()

API_KEY = load_api_key()
```

Getting datasets

In [6]:
```
downloaded_paths = {
    "asbestos": r"data\asbestos",
    "crack": r"data\crack",
    "house": r"data\house",
    "paint": r"data\paint",
    "mold": r"data\mold",
}
```

In [7]:
```
from pathlib import Path
import pandas as pd

def count_files(folder, exts):
    folder = Path(folder)
    if not folder.exists():
        return 0
    return sum(1 for f in folder.rglob("*") if f.is_file() and f.suffix.lower()

def count_dataset(base_path):
    base = Path(base_path)

    return {
        "train_images": count_files(base / "train" / "images", {".jpg", ".jpeg",
        "valid_images": count_files(base / "valid" / "images", {".jpg", ".jpeg",
        "test_images": count_files(base / "test" / "images", {".jpg", ".jpeg", "

        "train_labels": count_files(base / "train" / "labels", {".txt"}),
        "valid_labels": count_files(base / "valid" / "labels", {".txt"}),
        "test_labels": count_files(base / "test" / "labels", {".txt"}),
    }

summary = []

for name, path in downloaded_paths.items():
    stats = count_dataset(path)
    stats["dataset"] = name
    summary.append(stats)

df_summary = pd.DataFrame(summary)
df_summary
```

Out[7]:

	train_images	valid_images	test_images	train_labels	valid_labels	test_labels	datase
0	932	266	133	932	266	133	asbestc
1	3717	200	112	3717	200	112	crac
2	3008	0	562	3008	0	562	hous
3	0	0	67	0	0	67	pair
4	1416	0	0	1416	0	0	mol

Dataset Characteristics

The datasets show significant differences in size and structure.

- The crack and house datasets are substantially larger than the others
- The paint dataset is very small and only contains test samples
- The mold dataset only contains training data
- The house dataset lacks a validation split

This indicates strong heterogeneity across datasets.

Implications

These differences have several implications:

- Larger datasets (crack, house) will dominate model learning
- Missing splits require careful handling during training and evaluation
- Small datasets (paint) may not contribute significantly to training
- The model must generalize across datasets with inconsistent structures

This variability reflects realistic conditions and is relevant for evaluating robustness.

Analysis 1:

Make sure to add some explanation of what you are doing in your code. This will help you and whoever will read this a lot in following your steps.

Total sizes + ratios

In [9]:
```
df = df_summary.copy()

df["total_images"] = (
    df["train_images"] +
    df["valid_images"] +
    df["test_images"]
)

df["train_ratio"] = df["train_images"] / df["total_images"]
df["valid_ratio"] = df["valid_images"] / df["total_images"]
df["test_ratio"] = df["test_images"] / df["total_images"]

df
```

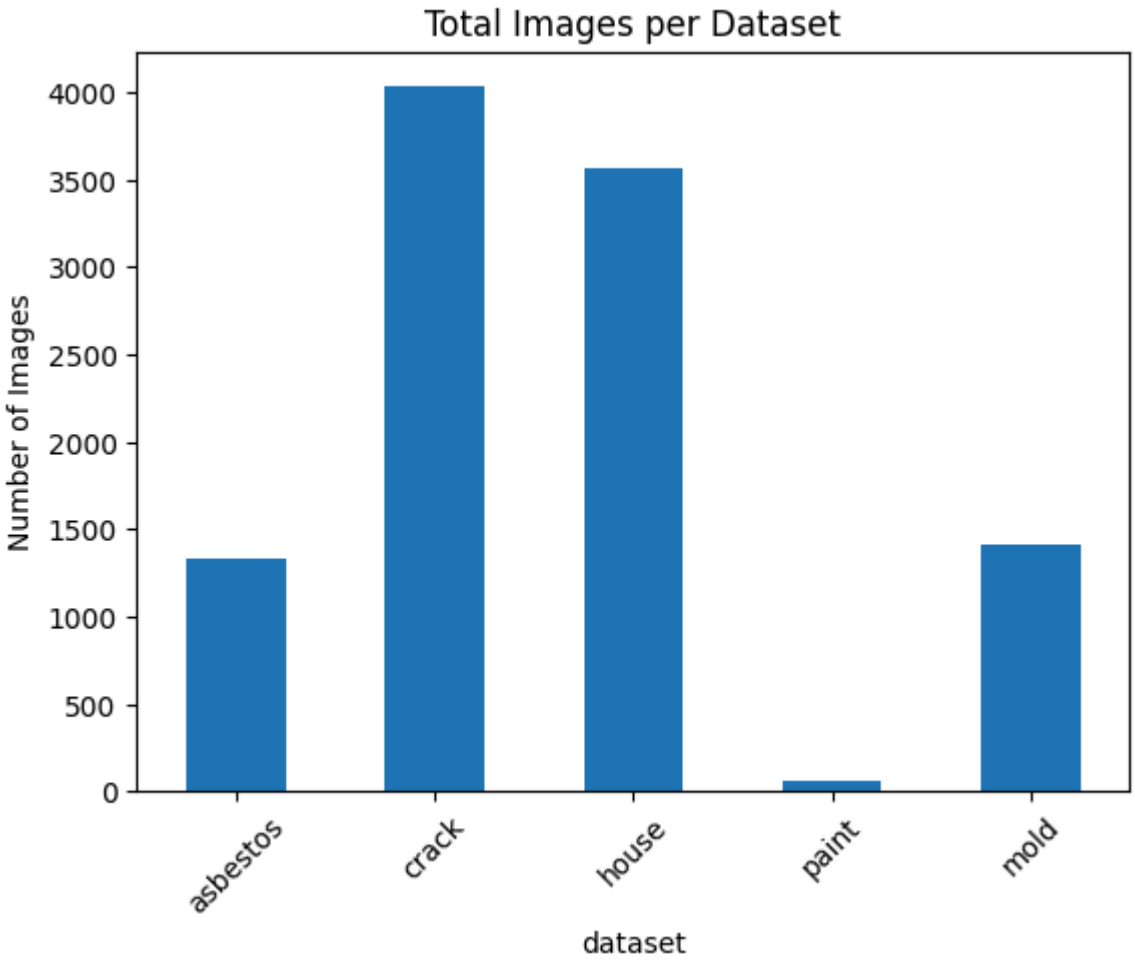
Out[9]:

	train_images	valid_images	test_images	train_labels	valid_labels	test_labels	datase
0	932	266	133	932	266	133	asbestc
1	3717	200	112	3717	200	112	crac
2	3008	0	562	3008	0	562	hous
3	0	0	67	0	0	67	pair
4	1416	0	0	1416	0	0	mol

Dataset size

In [10]:

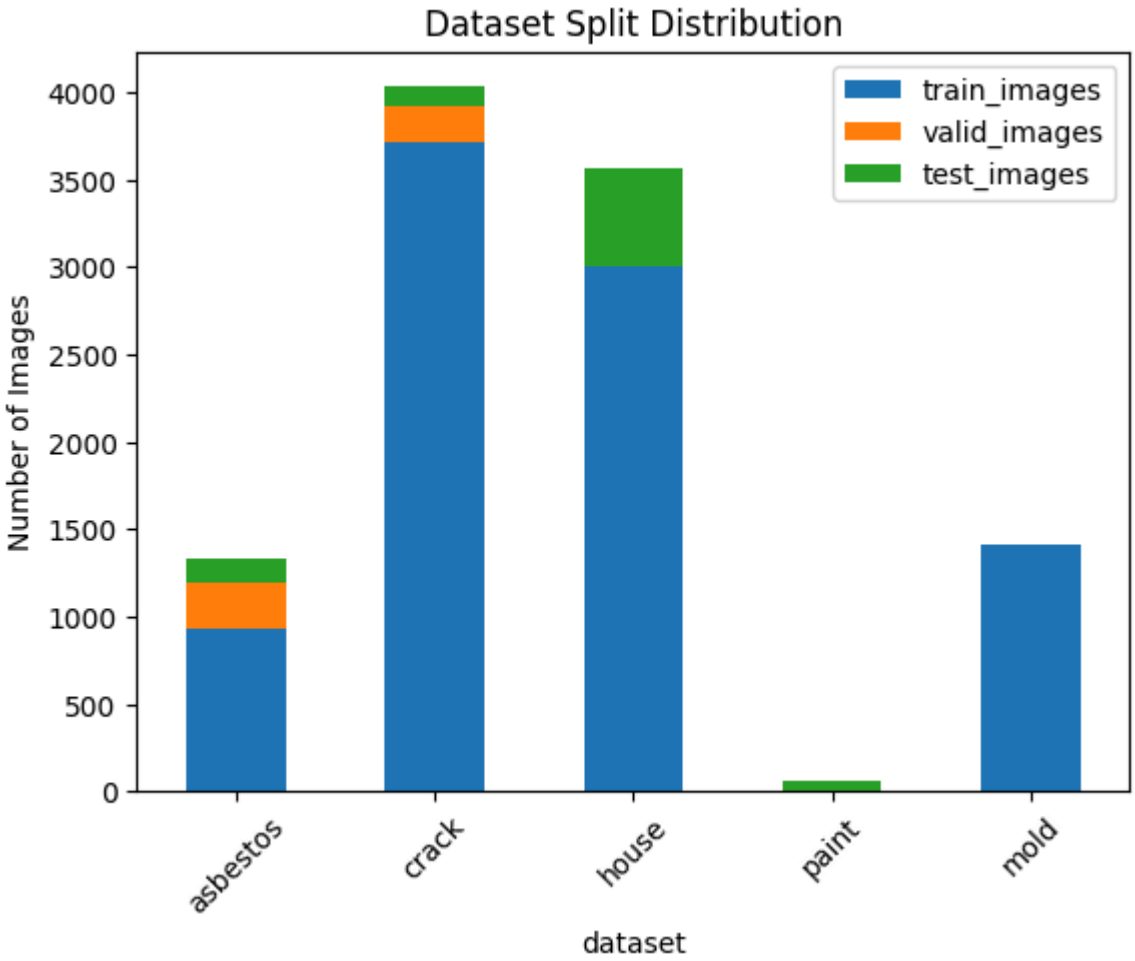
```
plt.figure()
df.set_index("dataset")["total_images"].plot(kind="bar")
plt.title("Total Images per Dataset")
plt.ylabel("Number of Images")
plt.xticks(rotation=45)
plt.show()
```



split distribution

In [11]:
```
df_split = df.set_index("dataset")[["train_images", "valid_images", "test_images

plt.figure()
df_split.plot(kind="bar", stacked=True)
plt.title("Dataset Split Distribution")
plt.ylabel("Number of Images")
plt.xticks(rotation=45)
plt.show()
```

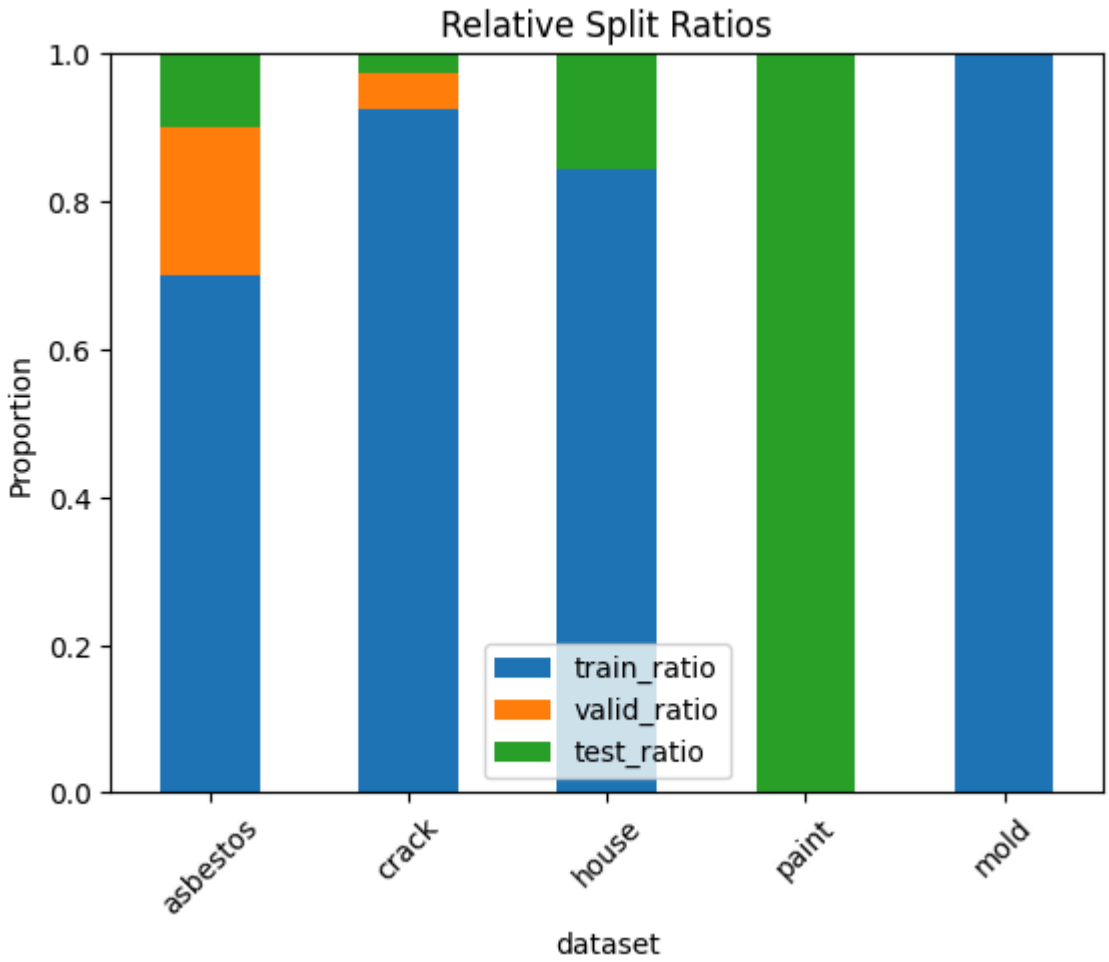
```
<Figure size 640x480 with 0 Axes>
```



Split ratio comparison

In [12]:
```
df_ratio = df.set_index("dataset")[["train_ratio", "valid_ratio", "test_ratio"]]

plt.figure()
df_ratio.plot(kind="bar", stacked=True)
plt.title("Relative Split Ratios")
plt.ylabel("Proportion")
plt.xticks(rotation=45)
plt.show()
```

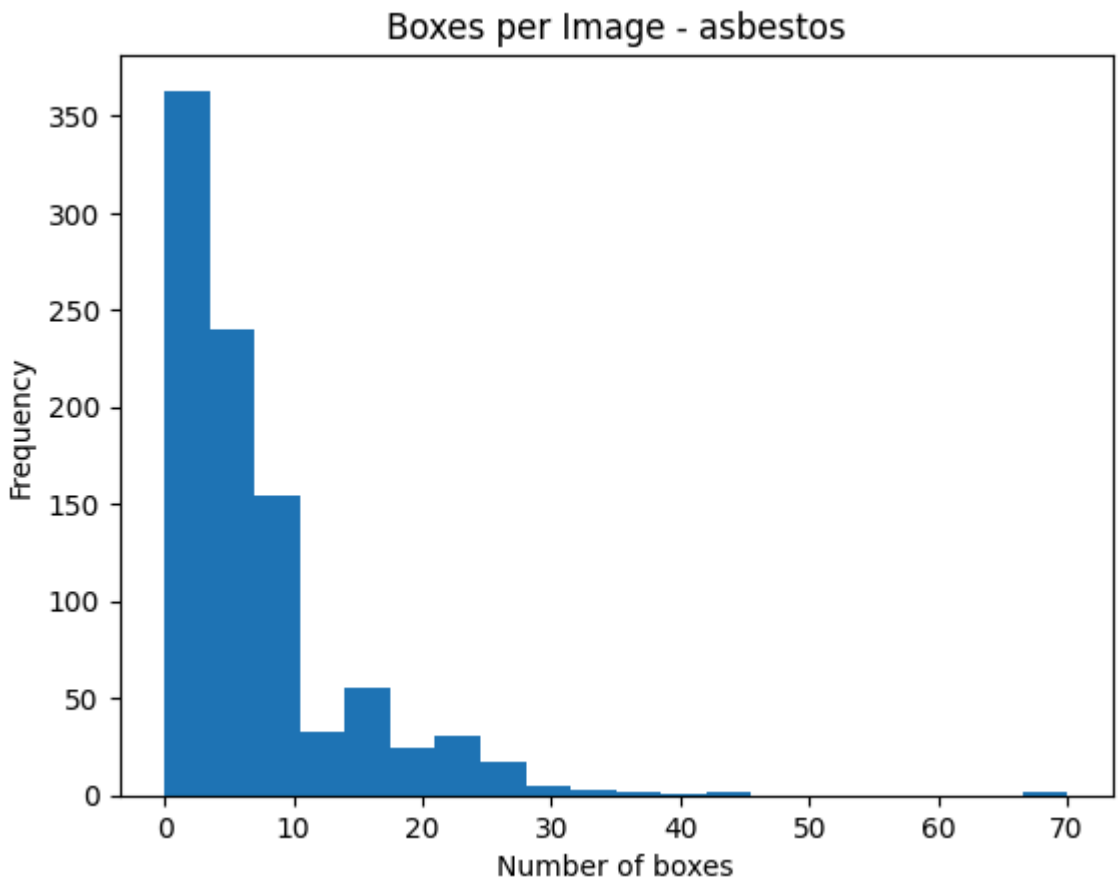
```
<Figure size 640x480 with 0 Axes>
```

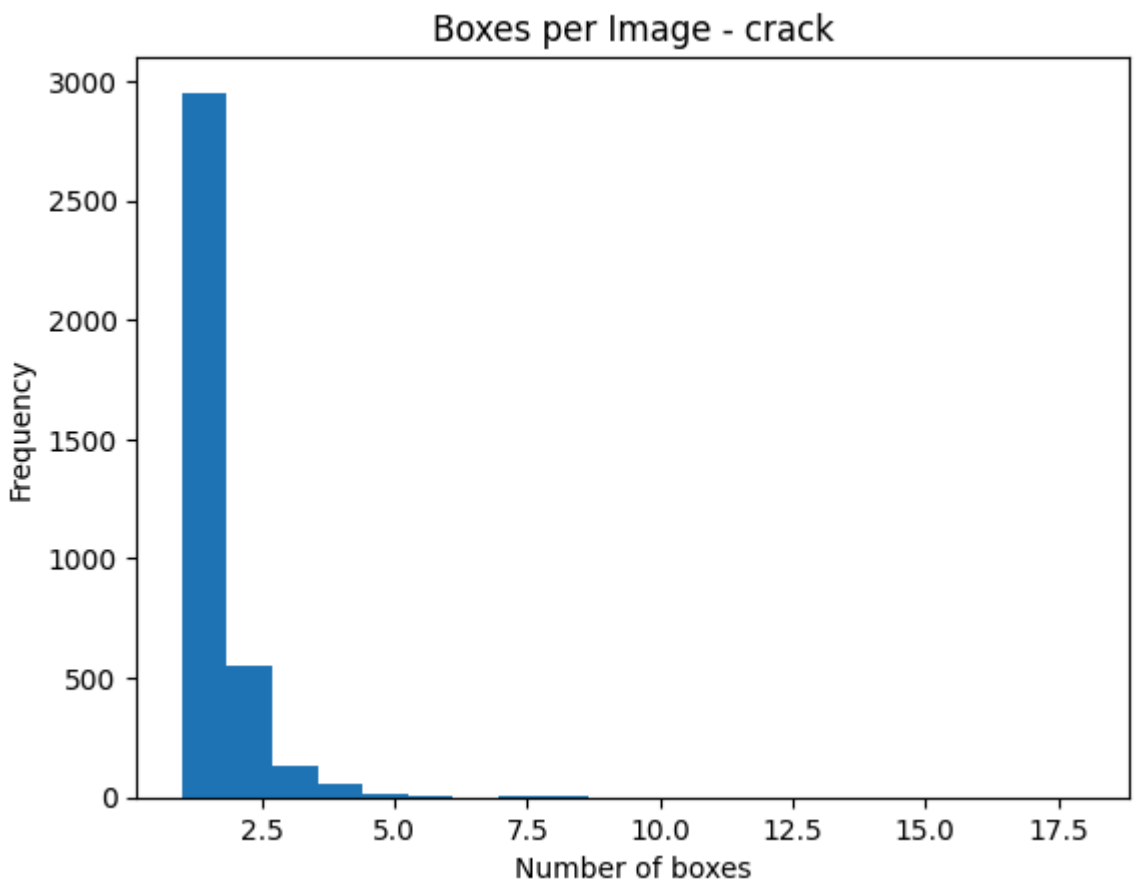


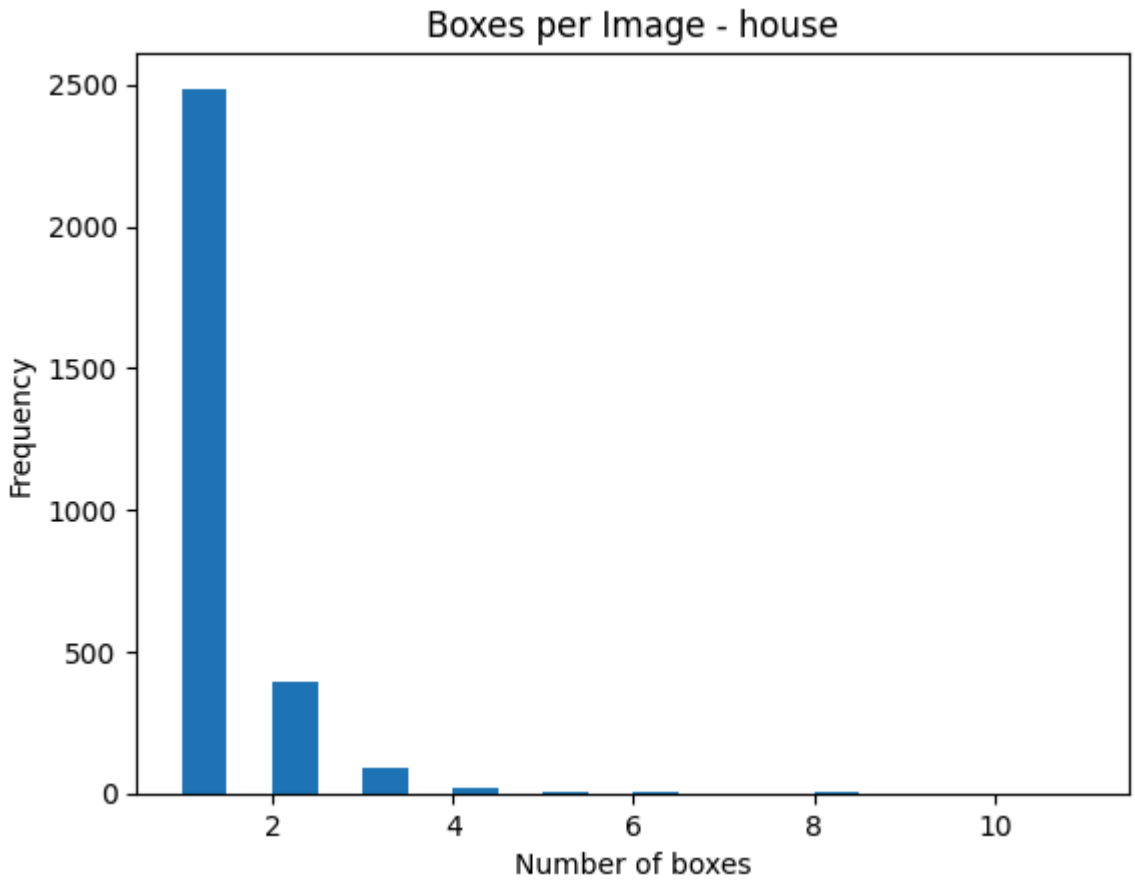
Annotation density

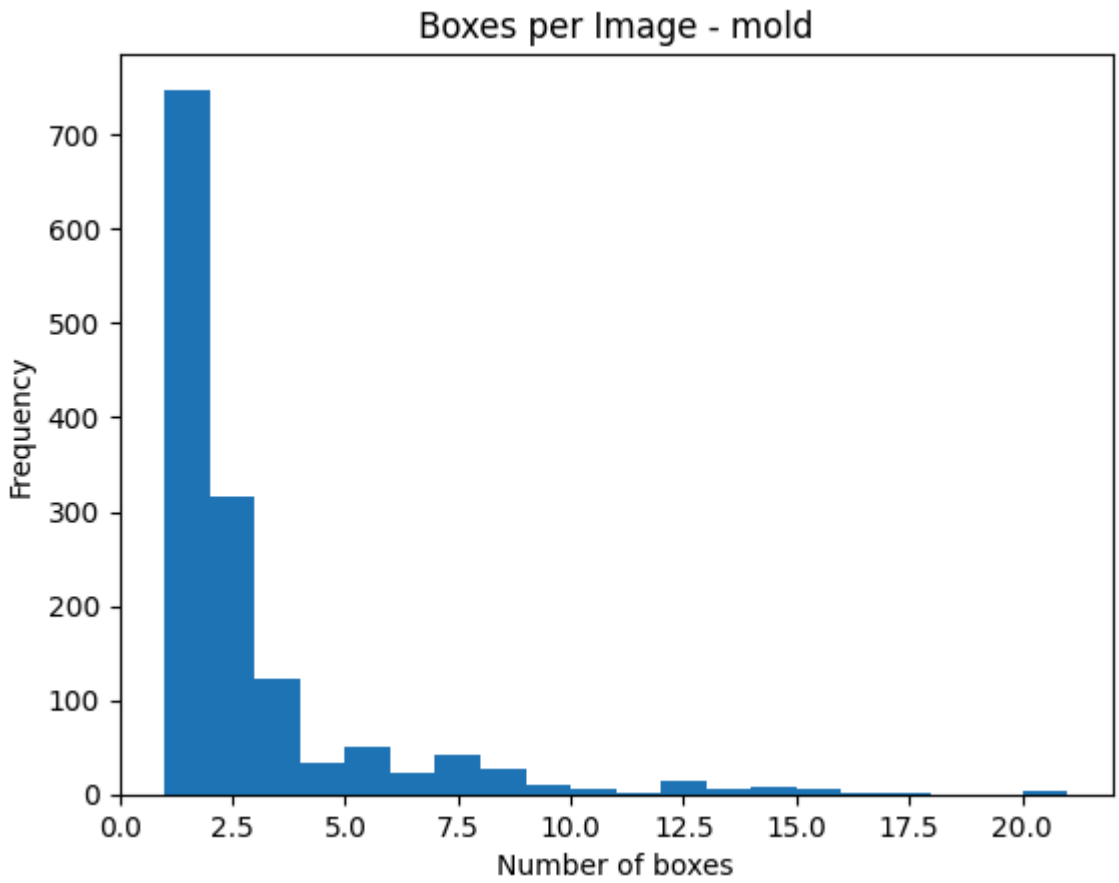
In [14]:
```
def boxes_per_image(path):
    counts = []
    path = Path(path)

    for file in path.rglob("*.txt"):
        with open(file) as f:
            counts.append(len(f.readlines()))
    return counts

for name, path in downloaded_paths.items():
    label_path = Path(path) / "train" / "labels"

    if not label_path.exists():
        continue

    counts = boxes_per_image(label_path)

    if len(counts) == 0:
        continue

    plt.figure()
    plt.hist(counts, bins=20)
    plt.title(f"Boxes per Image - {name}")
    plt.xlabel("Number of boxes")
```

```
plt.ylabel("Frequency")
plt.show()
```









Average boxes per image

In [17]:
```
avg_boxes = {}

for name, path in downloaded_paths.items():
    label_path = Path(path) / "train" / "labels"

    if not label_path.exists():
        avg_boxes[name] = 0
        continue

    counts = boxes_per_image(label_path)

    avg_boxes[name] = sum(counts) / len(counts) if counts else 0

df_boxes = pd.DataFrame.from_dict(avg_boxes, orient="index", columns=["avg_boxes
df_boxes
```

Out[17]:

	avg_boxes_per_image
asbestos	7.041845
crack	1.316384
house	1.245013
paint	0.000000
mold	2.439972

classes count

In [18]:
```
import yaml

class_counts = {}

for name, path in downloaded_paths.items():
    yaml_path = Path(path) / "data.yaml"

    if yaml_path.exists():
        with open(yaml_path) as f:
            data = yaml.safe_load(f)
            class_counts[name] = len(data.get("names", []))
    else:
        class_counts[name] = 0

df_classes = pd.DataFrame.from_dict(class_counts, orient="index", columns=["num_
df_classes
```

Out[18]:

	num_classes
asbestos	4
crack	1
house	4
paint	1
mold	5

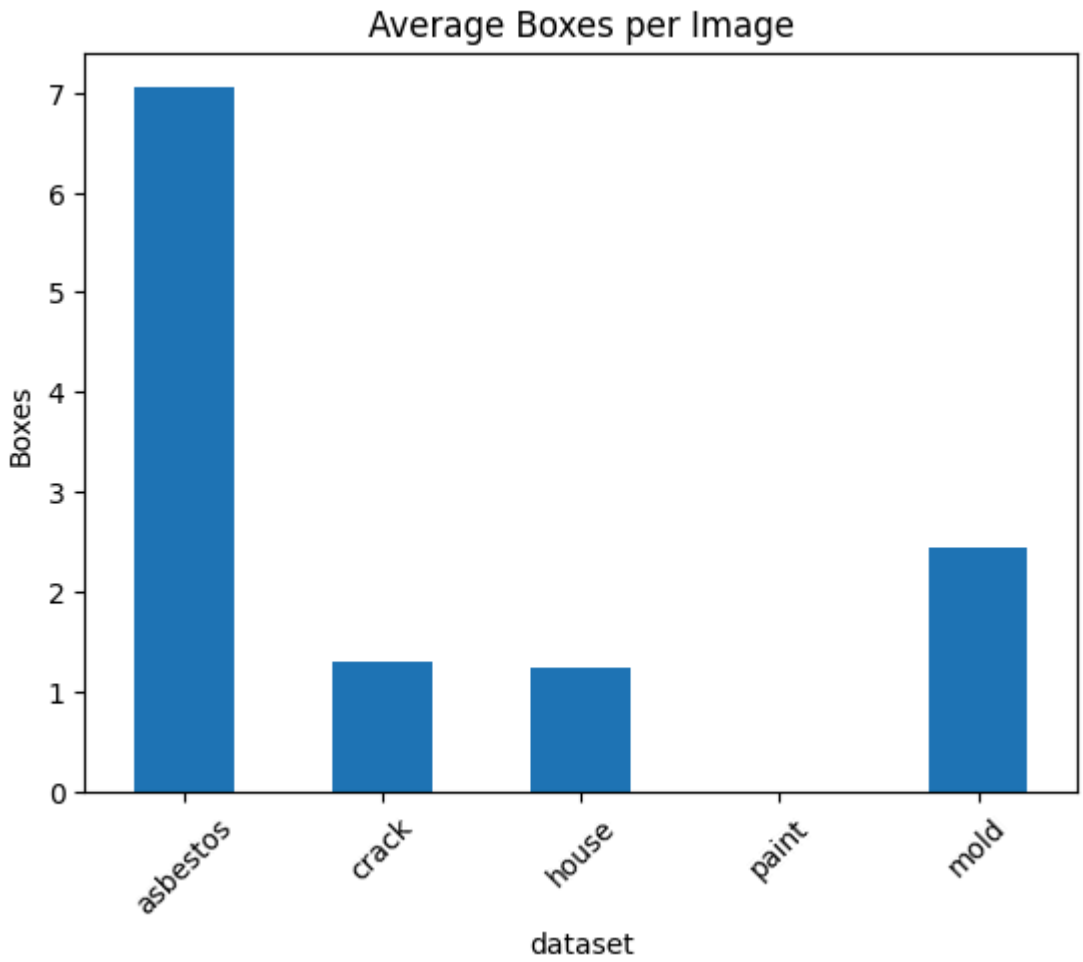
combining key metrics

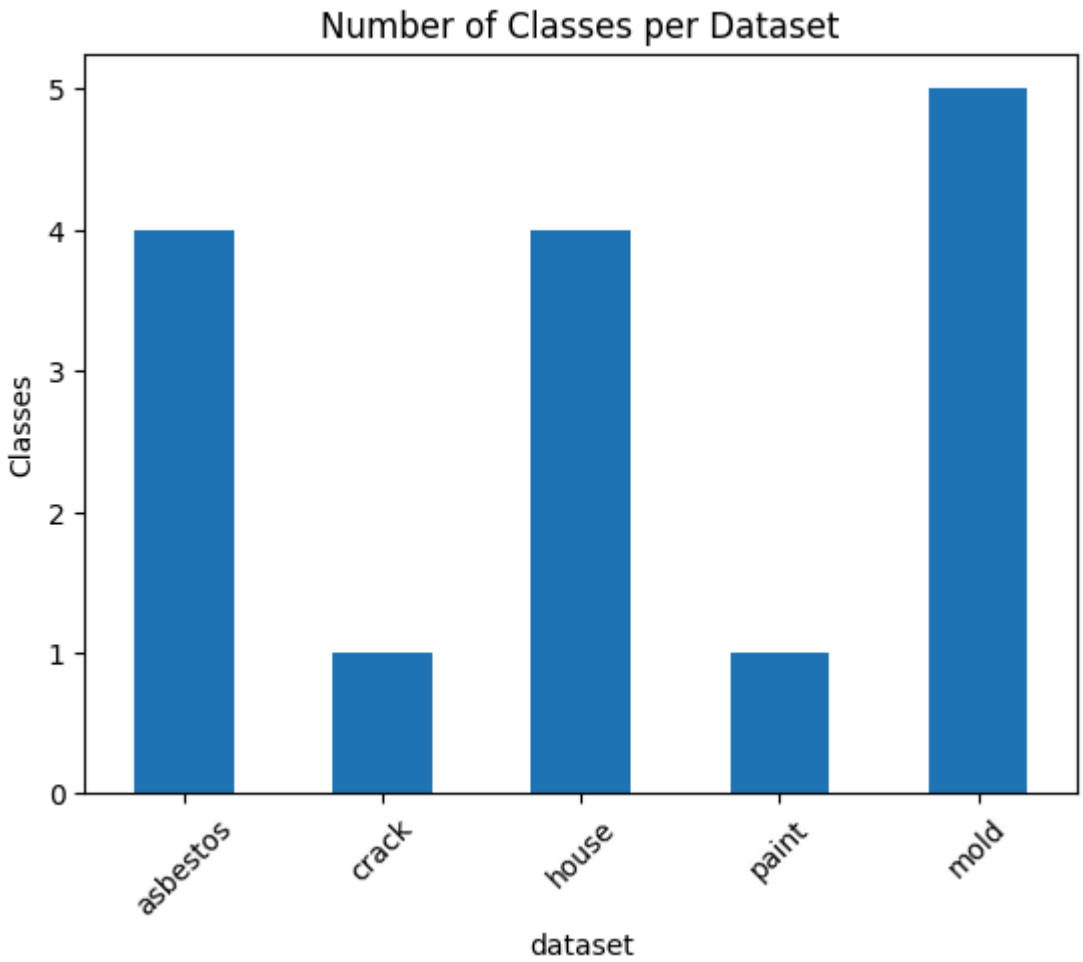
In [19]:
```
df_analysis = df.set_index("dataset").join(df_boxes).join(df_classes)
df_analysis
```

Out[19]:

dataset	train_images	valid_images	test_images	train_labels	valid_labels	test_labels
asbestos	932	266	133	932	266	133
crack	3717	200	112	3717	200	112
house	3008	0	562	3008	0	562
paint	0	0	67	0	0	67
mold	1416	0	0	1416	0	0

In [20]:
```
plt.figure()
df_analysis["avg_boxes_per_image"].plot(kind="bar")
plt.title("Average Boxes per Image")
plt.ylabel("Boxes")
plt.xticks(rotation=45)
plt.show()

plt.figure()
df_analysis["num_classes"].plot(kind="bar")
plt.title("Number of Classes per Dataset")
plt.ylabel("Classes")
plt.xticks(rotation=45)
plt.show()
```


Average Boxes per Image
Boxes
0
1
2
3
4
5
6
7
asbestos
crack
house
paint
mold
dataset


Number of Classes per Dataset
Classes
0
1
2
3
4
5
asbestos
crack
house
paint
mold
dataset

Relevance to the Research Objective

The identified dataset characteristics directly impact the comparison between YOLOv8 and LLaVA.

YOLOv8 relies on structured bounding box annotations and is therefore sensitive to:

- dataset imbalance
- annotation consistency
- availability of training and validation splits

In contrast, LLaVA operates as a vision-language model and may be less dependent on strict annotation structures, potentially allowing it to generalize better across heterogeneous datasets.

This difference makes the dataset variability particularly relevant for evaluating the robustness of both approaches.

Implications for Methodology

Based on the EDA, the following steps are required:

- Standardization of labels across datasets
- Handling of missing dataset splits
- Careful interpretation of results due to dataset imbalance

These steps ensure a fair and meaningful comparison between YOLOv8 and LLaVA.

In [1]:
```
!jupyter nbconvert --to html "Exploratory Data Analysis.ipynb"
```

```
[NbConvertApp] Converting notebook Exploratory Data Analysis.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 9 image(s).
[NbConvertApp] Writing 546956 bytes to Exploratory Data Analysis.html
```

In [ ]:

In [ ]:

In [ ]: